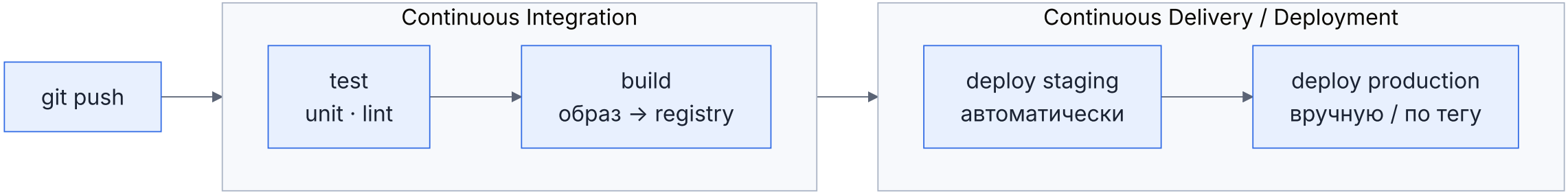
CI/CD. Обзор GitLab. Установка GitLab в Kubernetes

Занятие 07 · День 4, утро

Как было без CI/CD

- «У меня на машине работает»
- Сборка руками, релиз раз в месяц, ночь перед релизом — общий сбор
- Ошибки находятся **поздно**: интеграция кода нескольких разработчиков — боль
- Деплой — по инструкции в wiki (устаревшей)
- Откат — «а кто помнит, что было до этого?»

CI, CD и CD



Практика	Суть
Continuous Integration	каждый push → автоматическая сборка и тесты; ошибки — за минуты
Continuous Delivery	любой успешный коммит **готов** к выкатке; выкатка — по кнопке
Continuous Deployment	успешный коммит **автоматически** уходит в production

Пайплайн: основные понятия

- **Pipeline** — весь процесс для одного коммита/тега/MR
- **Stage** — этап: `test` → `build` → `deploy`; этапы идут по порядку
- **Job** — задача внутри этапа; jobs одного этапа идут **параллельно**
- **Runner** — агент, который выполняет jobs
- **Artifact** — результат job (отчёт, бинарник), передаётся дальше
- **Environment** — куда деплоим: staging, production

> Хороший пайплайн: быстрый (минуты), воспроизводимый, и **красный** при любой проблеме.

Как измерить: метрики DORA

Метрика	Вопрос	Элита
Deployment frequency	как часто выкатываем	по требованию, много раз в день
Lead time for changes	от коммита до production	< 1 дня
Change failure rate	доля неудачных выкаток	низкая
Time to restore	как быстро восстанавливаемся	< 1 часа

CI/CD — главный инструмент для улучшения всех четырёх.

Инструменты CI/CD

Инструмент	Особенность
GitLab CI/CD	встроен в GitLab: код, MR, registry, CI — в одном месте
GitHub Actions	встроен в GitHub, большой маркетплейс
Jenkins	ветеран, плагины на всё, требует сопровождения
Argo CD, Flux	**GitOps**-доставка в Kubernetes (pull-модель)
Tekton, Argo Workflows	CI-конвейеры как объекты Kubernetes
TeamCity, Bamboo, Azure DevOps	коммерческие

GitLab — DevSecOps-платформа

- **Код**: Git-репозитории, Merge Requests, code review
- **Планирование**: issues, boards, milestones, epics*
- **CI/CD**: пайплайны, runners, environments, релизы
- **Пакеты**: Container Registry, Package Registry, Helm-репозиторий
- **Безопасность**: SAST, Secret Detection, Dependency Scanning*
- **Эксплуатация**: интеграция с Kubernetes (agent), feature flags

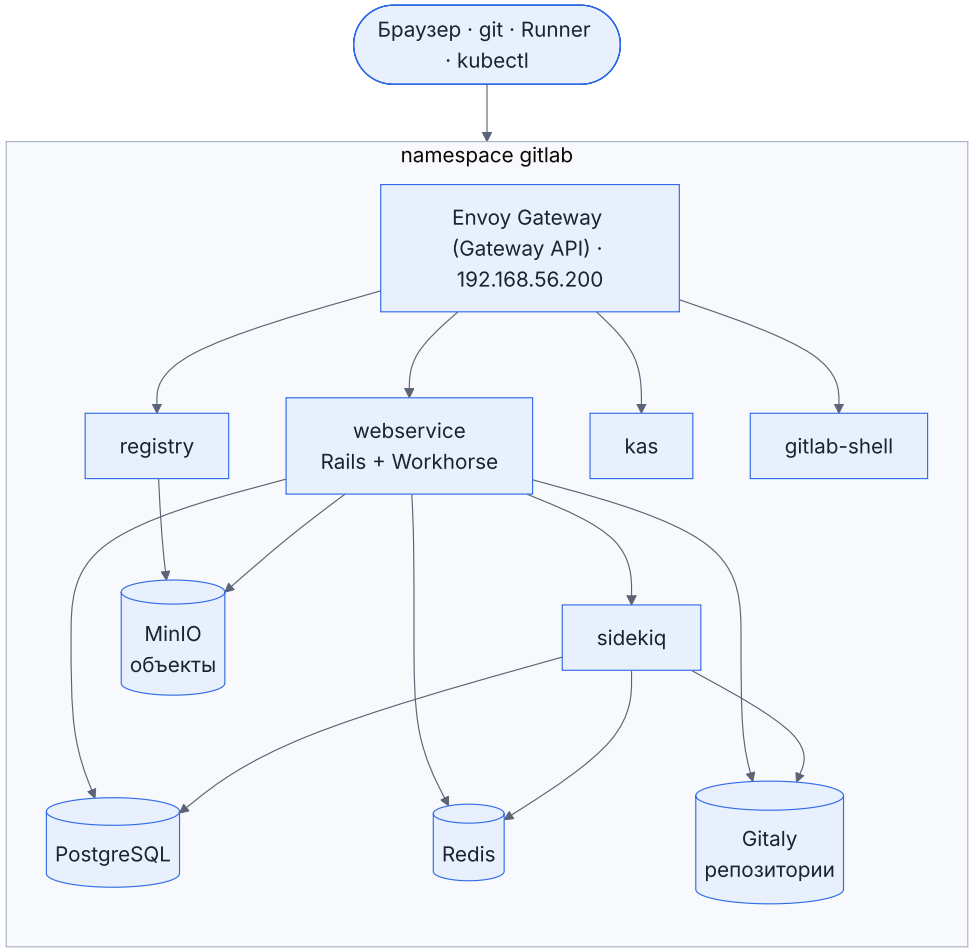
Редакция	
CE (Community Edition)	бесплатная, open source — ставим её
EE (Premium, Ultimate)	расширенные функции (*), подписка

Варианты GitLab

Вариант	Как	Когда
GitLab.com (SaaS)	регистрация	быстро, без сопровождения
Linux package (Omnibus)	`apt install gitlab-ce` на ВМ	классика self-managed
Docker	`gitlab/gitlab-ce`	тесты, маленькие команды
Helm-чарт `gitlab/gitlab`	в Kubernetes, cloud native	масштабирование, k8s-платформа
GitLab Operator	оператор поверх чарта	OpenShift, управление жизненным циклом

Сегодня — **Helm-чарт** в нашем кластере.

Архитектура GitLab в Kubernetes



Компоненты

Компонент	Роль
webservice	Rails-приложение (UI, API) + **Workhorse** (загрузки, git по HTTP)
sidekiq	фоновые задачи: почта, пайплайны, вебхуки
gitaly	хранение Git-репозиториев (StatefulSet + PVC)
gitlab-shell	`git` по SSH
registry	Container Registry (образы Docker/OCI)
kas	сервер для GitLab agent for Kubernetes
PostgreSQL, Redis, MinIO	БД, кэш/очереди, объектное хранилище
toolbox	бэкапы, rake-задачи, консоль
Envoy Gateway	вход: Gateway API, HTTP/SSH

> В production PostgreSQL, Redis и объектное хранилище выносят **за пределы** кластера.

Что нового во входе: Gateway API

- С **GitLab 19.0** чарт по умолчанию использует **Gateway API + Envoy Gateway**
- Встроенный **NGINX Ingress устарел** и будет удалён в GitLab 20.0
- Чарт сам ставит CRD Gateway API и Envoy Gateway при первой установке
- Envoy получает Service `LoadBalancer` → у нас IP от **MetalLB**

```yaml
global:
  gatewayApi:
    enabled: true          # по умолчанию с 19.0
    installEnvoy: true
```

Helm за 3 минуты

- **Пакетный менеджер** для Kubernetes: «apt для кластера»
- **Чарт** — шаблоны манифестов + `values.yaml` (параметры по умолчанию)
- **Релиз** — установленный экземпляр чарта, с историей ревизий

```
helm repo add gitlab https://charts.gitlab.io/
helm search repo gitlab/gitlab
helm show values gitlab/gitlab > all-values.yaml   # ~ тысячи строк параметров
helm upgrade --install <релиз> <чарт> -n <ns> -f my-values.yaml
helm list -A ; helm history <релиз> -n <ns> ; helm rollback <релиз> 1 -n <ns>
```

Сейчас Helm — **инструмент установки**; на 5-й день напишем свой чарт.

ДЕМО Установка Helm на master-1

```
HELM_VERSION=v4.3.1
ARCH=$(dpkg --print-architecture)                # amd64 или arm64
curl -fsSL "https://get.helm.sh/helm-${HELM_VERSION}-linux-${ARCH}.tar.gz" \
  | tar xz -C /tmp
sudo install -m 0755 /tmp/linux-${ARCH}/helm /usr/local/bin/helm
helm version
echo 'source <(helm completion bash)' >> ~/.bashrc
```

- **Helm 4** — текущая основная версия (ноябрь 2025); Helm 3 получает только исправления
- В Helm 4: server-side apply для новых релизов, `--atomic` → `--rollback-on-failure`, `--force` → `--force-replace`

Требования к кластеру

- Kubernetes с **StorageClass по умолчанию** → `local-path` ✓
- **LoadBalancer** для Envoy → MetalLB, IP `192.168.56.200` ✓
- **DNS-имена**: `gitlab.<домен>`, `registry.<домен>`, `kas.<домен>`, `minio.<домен>` → `*.192.168.56.200.nip.io` ✓
- **Ресурсы**: минимум ~4 CPU и 6–8 GB RAM **свободных** на workers

```
kubectl get sc
kubectl top nodes
kubectl -n metallb-system get ipaddresspool
nslookup gitlab.192.168.56.200.nip.io
```

values.yaml для стенда (1/2)

```yaml
global:
  edition: ce
  time_zone: Asia/Almaty
  hosts:
    domain: 192.168.56.200.nip.io     # gitlab.<domain>, registry.<domain>, …
    https: false                      # учебный стенд — без TLS
  gatewayApi:
    enabled: true
    installEnvoy: true
    configureCertmanager: false
  kas:
    enabled: true

gatewayApiResources:
  gateway:
    protocol: HTTP
    infrastructure:
      annotations:
        metallb.universe.tf/loadBalancerIPs: 192.168.56.200
  envoy:
    clientTrafficPolicySpec:
      path:
        escapedSlashesAction: KeepUnchanged
```

values.yaml для стенда (2/2)

```yaml
installCertmanager: false
nginx-ingress:
  enabled: false           # устаревший встроенный ingress
prometheus:
  install: false
gitlab-runner:
  install: false           # runner поставим сами на отдельную ВМ (занятие 09)

gitlab:
  webservice:
    minReplicas: 1
    maxReplicas: 1
    resources: { requests: { cpu: 300m, memory: 1.5G } }
  sidekiq:
    minReplicas: 1
    maxReplicas: 1
    resources: { requests: { cpu: 200m, memory: 800M } }
  gitlab-shell: { minReplicas: 1, maxReplicas: 1 }
  kas:          { minReplicas: 1, maxReplicas: 1 }
  gitlab-exporter: { enabled: false }
registry:
  hpa: { minReplicas: 1, maxReplicas: 1 }
```

ДЕМО Установка

```
helm repo add gitlab https://charts.gitlab.io/
helm repo update

helm upgrade --install gitlab gitlab/gitlab \
  --namespace gitlab --create-namespace \
  -f values.yaml \
  --timeout 20m

kubectl -n gitlab get pods -w
```

Порядок старта: PostgreSQL, Redis, MinIO, Gitaly → **migrations** (Job) → webservice, sidekiq → registry, kas, shell. Около **10–15 минут**.

ДЕМО Проверка и первый вход

```
kubectl -n gitlab get pods
kubectl -n gitlab get pvc
kubectl -n gitlab get gateway,httproute
kubectl get svc -A | grep -i envoy                # EXTERNAL-IP 192.168.56.200

# пароль root
kubectl -n gitlab get secret gitlab-gitlab-initial-root-password \
  -o jsonpath='{.data.password}' | base64 -d ; echo
```

Браузер: http://gitlab.192.168.56.200.nip.io → `root` / пароль выше

Первые настройки администратора

- Сменить пароль `root` (Edit profile → Password)
- **Admin → Settings → General → Sign-up restrictions**: отключить самостоятельную регистрацию
- Создать **личного пользователя** и работать не под `root`
- **Admin → Settings → CI/CD**: проверить, что Container Registry включён
- Посмотреть **Admin → Monitoring → Health check**

```
# консоль администратора (rake-задачи, бэкап)
kubectl -n gitlab exec -it deploy/gitlab-toolbox -- gitlab-rake gitlab:env:info
kubectl -n gitlab exec -it deploy/gitlab-toolbox -- backup-utility --help
```

Если что-то не так

Симптом	Где искать
Поды `Pending`	`describe pod` → нехватка памяти / PVC не создан (`kubectl get sc`)
`gitlab-migrations` Job падает	`kubectl -n gitlab logs job/…migrations…` — БД ещё не готова, Job перезапустится
webservice `0/2 Ready` долго	норма первые минуты: Rails прогревается; `kubectl top pods -n gitlab`
Envoy без EXTERNAL-IP	MetalLB: пул `gitlab` и аннотация с IP `.200`
Имя не резолвится	DNS-rebinding на роутере → запись в `hosts` на хосте
502 / 503	webservice ещё не Ready, `kubectl -n gitlab get pods`

Итоги

- **CI** — проверяем каждое изменение, **CD** — всегда готовы выкатить / выкатываем автоматически
- **GitLab** — код, MR, CI/CD, registry в одном продукте; ставим **CE**
- Cloud native GitLab = набор сервисов: webservice, sidekiq, gitaly, registry, kas…
- Вход — **Gateway API + Envoy Gateway** (с 19.0), IP — от MetalLB
- Helm: `repo add` → `upgrade --install -f values.yaml`

Практика 07

GitLab в вашем кластере

Задание — на платформе testlms.kz